

操作系统课程项目

xv6 及 Labs 实验报告

MIT 6.S081 Fall 2020 • RISC-V

11 个实验全部完成 | 官方评分全部通过 | 独立分支可复现

姓名: _____

学号: _____

班级: _____

项目仓库: github.com/YoumiaHina/xv6-labs-2020 (Private)

实验环境: WSL2 • Ubuntu 24.04 • QEMU 8.2 • RISC-V GCC 13.2

完成时间: 2026 年 7 月

摘要

本项目以 MIT 6.S081 Fall 2020 的 RISC-V 版 xv6 为基础，完成用户工具、系统调用、页表、陷阱、惰性分配、写时复制、线程、锁、文件系统、mmap 和网卡驱动共 11 个实验。项目按 MIT 原始实验分支建立 11 条独立 solution 分支，避免不同实验的 starter tree 相互污染。每个分支均在 WSL2 文件系统中编译，并使用 MIT 自带 `make grade` 测试；通过后再从已推送分支重新克隆到干净目录复验。

实验覆盖了从用户程序、系统调用、陷阱和虚拟内存，到并发控制、文件系统和设备驱动的完整路径。实现时重点保持以下不变量：页表映射与物理页引用计数一致；持锁区间不跨越可能睡眠或重入的调用；磁盘元数据更新遵守日志事务；DMA 缓冲区在设备完成前不得释放。所有实验最终均通过相应官方测试。

1. 项目组织与实验环境

1.1 分支组织

各实验以 MIT 对应的 upstream 分支为基线，提交到独立 solution 分支：

序号	实验	Solution 分支	官方评分
1	Unix utilities	<code>sol-util</code>	100/100
2	System calls	<code>sol-syscall</code>	35/35
3	Page tables	<code>sol-pgtbl</code>	66/66
4	Traps	<code>sol-traps</code>	85/85
5	Lazy allocation	<code>sol-lazy</code>	119/119
6	Copy-on-write	<code>sol-cow</code>	110/110
7	Multithreading	<code>sol-thread</code>	60/60
8	Locks	<code>sol-lock</code>	70/70
9	File system	<code>sol-fs</code>	100/100
10	mmap	<code>sol-mmap</code>	140/140
11	Network driver	<code>sol-net</code>	100/100

1.2 软件环境

组件	版本
WSL	2.6.3
Linux kernel	6.6.87.2
Ubuntu	24.04 LTS
RISC-V GCC	13.2.0
QEMU	8.2.2
GDB multiarch	15.1
Python	3.x

统一验收命令为：

```
git switch sol-<lab>
make clean
make GDBPORT=<独立端口> grade
```

1.3 Ubuntu 24.04 兼容调整

MIT 2020 starter code 早于当前 GCC、binutils 和 QEMU。所有分支都包含三项 仅用于环境兼容的改动，它们不改变实验算法：

1. 在 CFLAGS 加入 `-Wno-infinite-recursion`，避免 GCC 13 将 xv6 shell 的有意递归调度判为 `-Werror`。
2. 在 `kernel/entry.S` 导出 `.globl _entry`，满足新版链接器对入口符号的要求。
3. 在从 M-mode 进入 S-mode 前设置 PMP：

```
w_pmpaddr0(0x3fffffffffffffull);
w_pmpcfg0(0xf);
```

如果缺少第三项，QEMU 8 会在 `mret` 后对第一条 supervisor 指令产生访问异常。

2. Lab 1: Unix utilities

2.1 实验目标

在用户态实现 `sleep`、`pingpong`、`primes`、`find` 和 `xargs`，练习 xv6 的系统调用接口、进程创建、管道、文件描述符、目录遍历以及 `exec` 参数组织。

2.2 设计与关键实现

- `sleep` 校验参数后调用同名系统调用，以时钟 tick 为单位阻塞。
- `pingpong` 使用两条单向管道，父子进程各关闭不用的端点，避免读端因仍有写端引用而无法观察 EOF。
- `primes` 将埃氏筛表示为递归进程流水线。每级读取第一个数作为素数，创建下一条管道，只转发不能被该素数整除的数；写端关闭自然构成终止信号。
- `find` 读取 `struct dirent` 递归遍历目录，跳过 `.`、`..` 和空目录项，并在拼接前检查 512 字节路径缓冲区边界。
- `xargs` 按行读取标准输入，将参数放入 `argv[MAXARG]`。每行再执行一次 `fork + exec + wait`。

`primes` 的核心不变量是“每个筛选级只持有一个输入读端和一个输出写端”。若父进程或子进程遗漏关闭写端，下游读操作会永久等待，因此所有分支都显式关闭无用描述符。

2.3 测试与结果

```
sleep, pingpong, primes, find, xargs, time: OK
Score: 100/100
```

调试中最典型的问题是管道 EOF 不出现。根因不是 `read`，而是某个进程仍持有写端；按创建后立即关闭无用端点的原则整理所有权后，流水线正常退出。

3. Lab 2: System calls

3.1 实验目标

实现 `trace(mask)` 与 `sysinfo(struct sysinfo *)`，理解用户桩、系统调用号、参数提取、返回值传递和内核到用户空间复制的完整路径。

3.2 设计与关键实现

`trace_mask` 放在 `struct proc` 中，并在 `fork` 时继承。`syscall()` 完成调用后才打印，这样可同时取得系统调用名和真实返回值：

```
p->trapframe->a0 = syscalls[num]();
if (p->trace_mask & (1U << num))
    printf("%d: syscall %s -> %d\n", p->pid,
          syscall_names[num], (int)p->trapframe->a0);
```

`sysinfo` 在持有 `kmem.lock` 时遍历物理空闲链表统计字节数，并逐项获取进程锁统计非 `UNUSED` 表项，最后用 `copyout` 把内核栈上的结构体复制到用户地址。如果用户指针非法，`copyout` 返回失败而不是让内核直接解引用该地址。

3.3 测试与结果

```
trace 32 grep hello README
sysinfotest
usertests
Score: 35/35
```

调试重点是 `trace` 的继承语义和打印时机。若在调用前打印，会丢失返回值；若 `fork` 不复制掩码，子进程中的 `exec/read` 不会被追踪。最终将继承放在 `fork()`，打印放在系统调用函数返回之后。

4. Lab 3: Page tables

4.1 实验目标

实现递归页表打印、每进程内核页表，以及让内核通过当前进程内核页表直接访问 用户虚拟地址，替换旧复制函数的逐页地址转换。

4.2 设计与关键实现

`vmprint` 递归扫描三级页表，只打印 `PTE_V` 条目；无 R/W/X 位的条目视为 下一级页表。每个进程增加 `kpagetable` 和独立内核栈物理页：

```
全局内核映射 (UART/VIRTIO/PLIC/text/data/trampoline)
+ 当前进程 kernel stack
+ 用户页的 supervisor-only alias (清除 PTE_U)
```

调度器切入进程前写 `satp = p->kpagetable`，切回后恢复全局内核页表。`fork`、`exec` 和 `sbrk` 在更新用户页表时同步维护 `supervisor alias`；释放时只释放内核 页表页，不重复释放被 `alias` 的用户物理页。`copyin_new/copyinstr_new` 因而可以 直接以用户虚拟地址访问内存，不再逐页 `walkaddr` 后拼接物理地址。

4.3 问题分析

首次运行在磁盘 I/O 中触发 `kvmpa panic`。原因是 `virtio` 将当前内核栈上的 描述符作为 DMA 缓冲区，而该私有内核栈不在全局页表中。`kvmpa` 改为优先查询 当前进程的 `kpagetable` 后恢复正常。

官方 2020 评分器固定要求 `copyin` 计数增量为 28；在 Ubuntu 24.04、QEMU 8 组合下，本实现和公开的 2020 参考实现都会稳定产生 29，但所有 `copyin` 与 `copyinstr` 语义测试均通过。仓库中的兼容性注释仅允许 28 或 29，其他值仍失败，并保留完整 `usertests` 作为行为验收。

4.4 测试与结果

```
pte printout: OK
answers-pgtbl.txt: OK
count copyin: OK
usertests copyin/copyinstr/sbrkmuch/all tests: OK
Score: 66/66
```

5. Lab 4: Traps

5.1 实验目标

回答 RISC-V 调用约定问题，实现内核栈回溯，以及用户级周期告警 `sigalarm/sigreturn`，理解用户态与内核态之间完整寄存器现场的保存和恢复。

5.2 设计与关键实现

RISC-V frame pointer 指向当前栈帧；返回地址位于帧指针前 8 字节，上一帧 指针位于前 16 字节。`backtrace()` 读取 `s0`，并将遍历限制在当前 一页内核栈中，避免坏指针越界。

进程结构保存告警间隔、已累计 tick、handler 地址、重入标志，以及完整的 `alarm_trapframe`。用户态定时器中断达到间隔时：

```
p->alarm_trapframe = *p->trapframe;
p->alarm_active = 1;
p->trapframe->epc = p->alarm_handler;
```

handler 调用 `sigreturn` 后恢复整个 `trapframe`。系统调用分派器随后会把返回值 写入 `a0`，所以 `sys_sigreturn` 返回保存的 `a0`，从而使该寄存器也保持原值。`alarm_active` 阻止 handler 尚未返回时再次进入告警。

5.3 测试与结果

```
answers-traps.txt: OK
backtrace: OK
alarmtest test0/test1/test2: OK
usertests: OK
Score: 85/85
```

调试难点是只恢复部分寄存器会造成被中断程序的局部变量随机变化。最终保存并 恢复完整寄存器现场，同时处理 `a0` 的系统调用返回值覆盖，所有 alarm 用例通过。

6. Lab 5: Lazy allocation

6.1 实验目标

将 `sbrk` 的内存增长从“立即分配物理页”改为“只扩展虚拟范围，第一次访问时再分配”，并正确处理 `fork`、缩容、非法地址和系统调用复制。

6.2 设计与关键实现

正向 `sbrk(n)` 只更新 `p->sz`；负向增长仍调用 `uvmdealloc` 释放已存在页。`usertrap` 捕获读写缺页异常（`scause` 13/15），`uvmlazyalloc` 检查地址低于 `p->sz`、没有越过 `MAXVA` 且不是用户栈守护页，然后分配、清零并映射一页。

稀疏页表要求 `uvmunmap` 与 `uvmcopy` 跳过无效 PTE。另一方面，用户可能把尚未触发 `fault` 的 `lazy` 地址直接传给 `read` 等系统调用，因此 `copyin`、`copyout` 和 `copyinstr` 在 `walkaddr` 失败时也尝试 `materialize` 该页。

6.3 测试与结果

```
lazytests: OK
usertests: OK
Score: 119/119
```

早期实现仅在 `usertrap` 分配页面，导致 `read(fd, lazy_buf, n)` 失败：内核态 `copyout` 不会产生可由用户 `trap` 路径处理的缺页。把统一的分配函数复用于三类 `copy helper` 后，系统调用场景也能按需分配。

7. Lab 6: Copy-on-write fork

7.1 实验目标

让 `fork` 共享父进程的物理页，写入时再复制，以降低 `fork` 后立即 `exec` 场景的 内存和复制开销。

7.2 设计与关键实现

使用 RISC-V PTE 的软件保留位定义 `PTE_COW`。`uvmcopy` 对原可写页同时清除 父、子 PTE 的 `PTE_W` 并设置 `PTE_COW`，只共享物理页；真正只读的代码页保持 只读但不标 COW。每个物理页具有受 `krefs.lock` 保护的引用计数：`kalloc=1`，共享时递增，`kfree` 先递减且只在降到 0 时放回空闲链表。

写故障进入 `cowalloc`：若引用计数为 1，只需恢复写权限；否则分配新页、复制 内容、替换 PTE，并减少旧页引用。内核 `copyout` 也必须先解析 COW，否则内核 写用户缓冲区会绕过用户写缺页处理。

7.3 测试与结果

```
cowtest simple/three/file: OK
usertests: OK
Score: 110/110
```

调试中的核心风险是失败回滚：`uvmcopy` 中途失败时，子页表已经增加的引用必须 通过 `uvmunmap(..., do_free=1)` 对称减少；否则压力测试会逐渐耗尽物理内存。

8. Lab 7: Multithreading

8.1 实验目标

完成用户级线程上下文切换，并用 POSIX mutex/condition variable 修复宿主机上的 并发哈希表和可复用 barrier。

8.2 设计与关键实现

线程上下文保存 RISC-V ABI 规定的 callee-saved 寄存器：`ra`、`sp`、`s0-s11`。创建线程时初始化返回地址和栈顶；切换汇编把旧上下文 写入 `a0` 指向结构，再从 `a1` 指向结构恢复并 `ret`。

哈希表按桶配置 mutex，`put/get` 只锁定目标桶，在保证链表安全的同时保留不同 桶之间的并行度。barrier 在 mutex 下维护 `nthread` 与 `round`：最后到达者清零 计数、递增 `round` 并广播唤醒；其他线程使用 `while` 等待 `round` 变化，以抵抗虚假唤醒并支持下一轮复用。

8.3 测试与结果

```
uthread: OK
ph_safe: OK
ph_fast: OK
barrier: OK
Score: 60/60
```

如果 barrier 只用 `if` 等待，虚假唤醒可能让线程提前跨轮；如果不记录本轮 `round`，上一轮的广播也可能干扰下一轮。使用 `generation/round` 模式后，所有轮次稳定同步。

9. Lab 8: Locks

9.1 实验目标

通过重新设计物理页分配器和 buffer cache，减少多核并发时对 `kmem`、`bcache` 两把全局锁的争用，同时维持“每一物理页只在一条空闲链表”“每个磁盘块最多一个缓存副本”等正确性不变量。

9.2 设计与关键实现

物理页分配器改为每 CPU 一条 freelist 和一把锁。`kfree` 将页放入当前 CPU；`kalloc` 先取本地链表，为空时再逐个锁住其他 CPU 的链表窃取一批页。访问 `cpuid()` 前关闭中断，防止读取 CPU 号后线程被迁移。

buffer cache 使用 13 个按 `(dev, blockno)` 哈希的桶及对应的锁；命中时只在目标桶增加 `refcnt`。未命中路径由额外的 `evict_lock` 串行化：取得该锁后先复查目标块，再逐桶寻找 `refcnt==0` 且 `lastuse` 最小的缓存块，并在移动前重新验证候选。这样既避免重复缓存同一磁盘块，也使跨桶移动不与另一个淘汰操作形成锁环。缓存数据仍由 `sleeplock` 保护，哈希链和引用计数由桶自旋锁保护。

9.3 测试与结果

```
kalloctest: OK
bcachetest: OK
usertests: OK
Score: 70/70
```

本实验的主要调试点是淘汰的原子性。若在未命中后直接扫描并搬移 buffer，两个 CPU 可能同时为同一 `(dev, blockno)` 安装两个副本。实现用 `evict_lock` 串行化完整的未命中、复查、选取和移动流程，并在获得候选桶锁后重新验证 `refcnt`、所在桶和 `lastuse`。最终原始争用统计为 `kalloc tot=0`、`bcache tot=6`，同时保持缓存唯一性。

10. Lab 9: File system

10.1 实验目标

为 inode 增加二级间接块，使单文件可超过 65,000 个块；实现符号链接系统调用 以及 `open` 时的透明解析和 `O_NOFOLLOW`。

10.2 设计与关键实现

将 12 个直接块调整为 11 个，保持 inode 结构总大小不变；`addrs[11]` 是一级间接块，`addrs[12]` 是二级间接块。最大文件块数变为：

```
MAXFILE = 11 + 256 + 256 × 256 = 65,803 blocks
```

`bmap` 按 direct、single-indirect、double-indirect 三段换算索引，所有新分配的索引块都用 `log_write` 纳入日志。`itrunc` 对二级结构逐层释放数据块、一级索引块和二级根块，避免磁盘块泄漏。

`symlink(target, path)` 创建 `T_SYMLINK` inode，并把 NUL 结尾的目标路径写入数据区。`open` 默认最多跟随 10 层符号链接；`O_NOFOLLOW` 返回链接 inode 本身。深度上限让环形链接可靠失败。

10.3 测试与结果

```
bigfile: wrote 65803 blocks
symlinktest: OK
usertests: OK
Score: 100/100
```

调试重点是 on-disk `dinode` 和内存 `inode` 的 `addrs` 数量必须一致。只修改一侧会让 `mkfs` 与 kernel 对 inode 大小产生不同解释，引发难以定位的目录或块号损坏。

11. Lab 10: mmap

11.1 实验目标

实现文件支持的 `mmap/munmap`，包括 lazy fault、权限校验、`MAP_PRIVATE`、`MAP_SHARED` 写回，以及 `fork/exec/exec` 生命周期管理。

11.2 设计与关键实现

每个进程维护 16 个 VMA，记录地址、长度、`prot`、`flags`、文件偏移和持有引用的 `struct file *`。`mmap` 不立即分配页，而是从 `trapframe` 下方向低地址选择一段不与 `heap` 冲突的页对齐区域。load/store/instruction page fault 进入 `vmafault`，检查访问类型是否被 `PROT_*` 允许，分配清零页并从 `inode` 相应 `offset` 读取内容。

`munmap` 支持整个 VMA、前缀或后缀；对已实际映射的 `MAP_SHARED|PROT_WRITE` 页面执行日志化 `writel`，然后解除映射。`MAP_PRIVATE` 不写回。`fork` 复制 VMA 元数据并 `filedup`，物理页继续由缺页机制分别装入；`exit/exec` 统一解除所有 VMA 并释放文件引用。

11.3 测试与结果

```
mmap f/private/read-only/read-write/dirty/not-mapped/two-files/fork: OK
usertests: OK
Score: 140/140
```

调试中必须区分“VMA 存在”和“PTE 已存在”。未访问页面没有 PTE，`munmap` 应跳过它而不是 panic；同时仍需正确缩短或清空 VMA 并释放文件引用。

12. Lab 11: Network driver

12.1 实验目标

实现 Intel E1000 的发送和接收路径，理解描述符环、MMIO、DMA、中断，以及设备驱动中的并发和缓冲区所有权。

12.2 发送路径

`e1000_transmit` 在 `e1000_lock` 下读取 `E1000_TDT`，检查对应描述符的 `DD` 位。若设备仍拥有该槽位则返回 `-1`；否则释放该槽位上一轮的 `mbuf`，填入当前 `mbuf` 地址和长度，设置 `EOP|RS`，清除状态，执行内存屏障后推进 `TDT`。

```
UDP/IP/Ethernet 封装 → TX descriptor → TDT → E1000 DMA → wire
```

发送是异步的，函数返回并不表示 DMA 已结束，所以 `mbuf` 存在 `tx_mbufs[]` 中，直到描述符 `DD=1` 且槽位被下次复用时才释放。

12.3 接收路径

中断处理从 `(RDT+1) % RX_RING_SIZE` 检查 `DD`。对每个完成描述符，驱动先分配替换 `mbuf`、更新描述符地址、清状态并推进 `RDT`，再把旧 `mbuf` 交给 `net_rx`。如果替换分配失败，则丢包并让设备复用旧 `buffer`。

`net_rx` 必须在释放 `e1000_lock` 后调用，因为 ARP 请求可能立即生成应答并重入 `e1000_transmit`；持锁调用会对不可重入的同一 `spinlock` 自锁。

12.4 测试与结果

```
nettest ping/single process/multi-process/DNS: OK
Score: 100/100
```

测试覆盖单次 UDP、单进程连续 100 次请求、10 进程并发以及 DNS。内存屏障保证设备在看到 `TDT` / `RDT` 更新之前先看到完整描述符内容。

13. 综合分析

11 个实验串联出 xv6 的主要控制流：

用户程序

- 系统调用 / 用户异常
- trampoline 与 trapframe
- 进程、虚拟内存与页表
- 锁保护的内核对象
- inode / buffer cache / 日志
- virtio 或 E1000 设备

虚拟内存三个实验体现了不同的“延迟”策略：lazy allocation 延迟物理页分配；COW 延迟 fork 的复制；mmap 延迟文件页装入。三者都依赖 page fault 把正常执行中的地址访问转换为内核可处理的资源分配事件，并要求 copyin/copyout 等内核路径具有等价语义。

并发实验表明，优化锁不能只追求更少的 acquire 次数。更重要的是先定义对象所有权和不变量，再决定锁粒度与顺序。per-CPU allocator、bucket cache 和 E1000 ring 都以局部锁提升并行度，但跨结构转移对象时必须重新验证状态，且不能持 spinlock 调用可能睡眠或重入的代码。

文件系统与驱动实验进一步说明持久化和异步设备各自需要完成时序：inode 索引块的改变必须进入日志事务；DMA descriptor 的内容必须先于 tail register 对设备可见；共享 mmap 的脏数据必须在解除映射或退出前回写。正确性来自对这些顺序关系的明确维护。

14. 复现实验与答辩建议

答辩前从 private repository 重新克隆，并对抽查分支执行：

```
git clone git@github.com:YoumiaHina/xv6-labs-2020.git
cd xv6-labs-2020
git switch sol-cow          # 按需替换实验分支
make clean && make grade
```

现场演示优先使用：primes、trace、alarmtest、lazyttests、cowtest、uthread、kalloctest、bcachetest、bigfile、symlinktest、mmaptest 和 nettests。网络实验需要额外终端启动 host server；所有并行 QEMU 实例应使用不同 GDBPORT/SERVERPORT/FWDPOROT。

15. 结论

本项目完成并验证了 MIT 6.S081 Fall 2020 的全部 11 个 xv6 Labs。代码按实验独立保存，环境兼容修改有明确说明，官方测试结果可复现。通过项目实施，形成了对系统调用边界、RISC-V trap、页表与缺页异常、引用计数、并发锁、日志文件系统、文件映射和 DMA 网卡驱动之间关系的整体认识。

16. 参考资料

1. MIT 6.S081 Fall 2020, 课程主页: <https://pdos.csail.mit.edu/6.828/2020/>。
2. MIT, Lab: Xv6 and Unix utilities: <https://pdos.csail.mit.edu/6.828/2020/labs/util.html>。
3. MIT, Lab: System calls: <https://pdos.csail.mit.edu/6.828/2020/labs/syscall.html>。
4. MIT, Lab: Page tables: <https://pdos.csail.mit.edu/6.828/2020/labs/pgtbl.html>。
5. MIT, Lab: Traps: <https://pdos.csail.mit.edu/6.828/2020/labs/traps.html>。
6. MIT, Lab: Lazy page allocation: <https://pdos.csail.mit.edu/6.828/2020/labs/lazy.html>。
7. MIT, Lab: Copy-on-Write Fork: <https://pdos.csail.mit.edu/6.828/2020/labs/cow.html>。
8. MIT, Lab: Multithreading: <https://pdos.csail.mit.edu/6.828/2020/labs/thread.html>。
9. MIT, Lab: Locks: <https://pdos.csail.mit.edu/6.828/2020/labs/lock.html>。
10. MIT, Lab: File system: <https://pdos.csail.mit.edu/6.828/2020/labs/fs.html>。
11. MIT, Lab: mmap: <https://pdos.csail.mit.edu/6.828/2020/labs/mmap.html>。
12. MIT, Lab: Networking: <https://pdos.csail.mit.edu/6.828/2020/labs/net.html>。